Jaire Bartlett

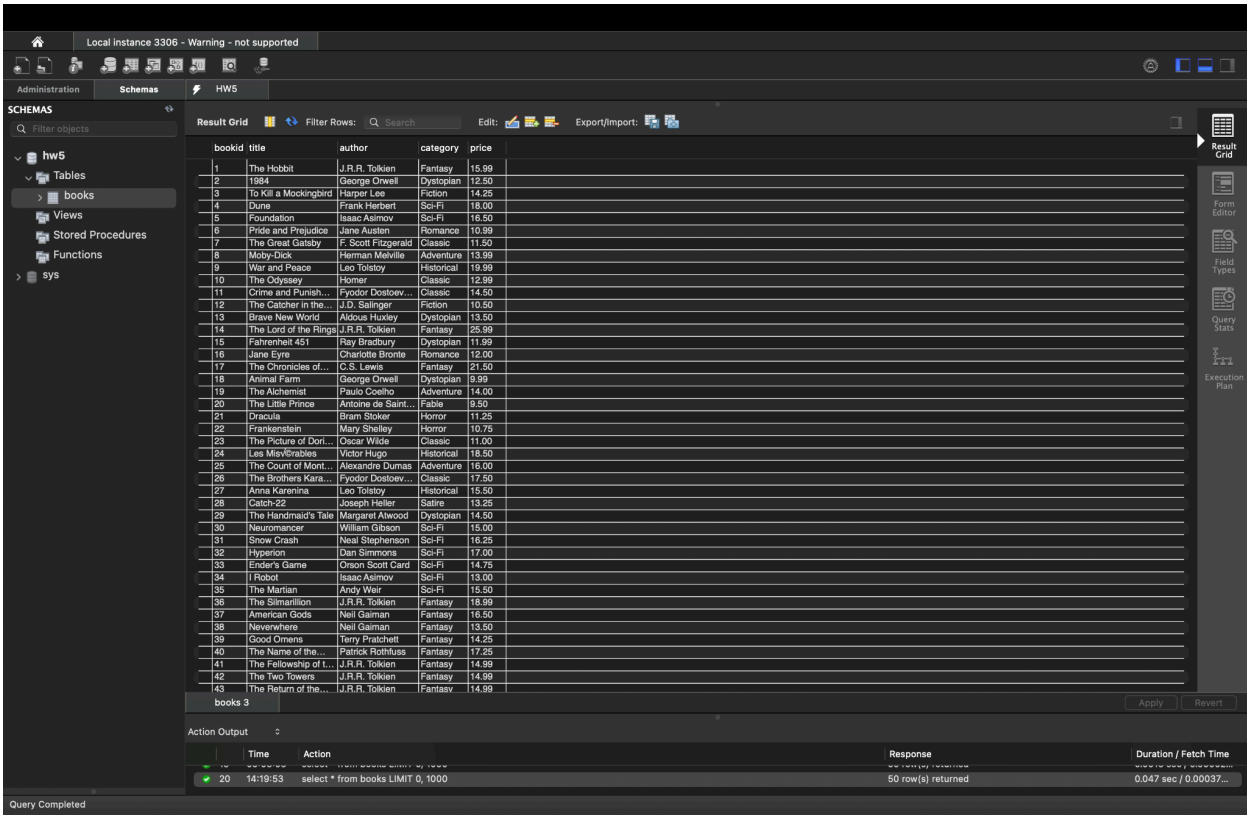
CSC 351

5 November 2026

Homework Assignment 5

Building a Simple Query String Web Server in Node.js

Table Creation:



Node.js Code Implementation & JSON Examples

Mysql.js:

```
// mysql.js - The Database Connection Pool Module
const mysql = require('mysql2/promise');


// Connection configuration settings
```

```
const dbConfig = {
   host: "localhost",
   user: "root",
   password: "password", // Replace with your actual MySQL root password
   database: "hw5",      // Replace with your actual database name
   waitForConnections: true,
   connectionLimit: 10,             // Recommended pool setting from your lecture
slides
   queueLimit: 0
};

// Create the connection pool
const pool = mysql.createPool(dbConfig);

// Export the pool so server.js can import it using require('./mysql')
module.exports = pool;
```

Server.js:

```
// server.js - Express Web Server for Book Queries
const express = require('express');
const app = express();
const port = 3000;

// Requires the exported pool object from the local 'mysql.js' file
const pool = require('./mysql');

//*** This root GET route handles extracting query variables and executing the query
app.get("/", async (req, res) => {

   // Extract strings received from the browser (matching Server 1 and Server 3 style)
   const category = req.query.category;
   const author = req.query.author;
   const format = req.query.format;

   // Define fixed query structures and parameters based on class material matching
rules
   let sql_query = "";
   let params = [];

   if (category) {
```

```
        sql_query = "SELECT * FROM books WHERE category = ?";
        params = [category];
    } else if (author) {
        sql_query = "SELECT * FROM books WHERE author = ?";
        params = [author];
    } else {
        // Default backup query if no specific filter is supplied
        sql_query = "SELECT * FROM books";
    }

    try {
        // SECURITY FIX: Await the promise from pool.execute using placeholders (Server
3 style)
        const [result, fields] = await pool.execute(sql_query, params);

        // Check the formatting parameter
        if (format === 'text') {

            //*** start creating the html body for the browser (matching Design Example
style)
            let html_body = "<HTML><STYLE>body{font-family:arial}</STYLE>";
            html_body += "<BODY>";
            html_body += "<h2>Query Match Results</h2>";
            html_body += "<TABLE BORDER=1 CELLPADDING=8>";

            //*** print column headings using fields array from mysql2
            html_body += "<TR>";
            for (let i = 0; i < fields.length; i++) {
                html_body += ("<TH style='background-color:#f2f2f2'>" +
fields[i].name.toUpperCase() + "</TH>");
            }
            html_body += "</TR>";

            //*** prints rows of table data
            for (let i = 0; i < result.length; i++) {
                const row = result[i];
                html_body += "<TR>";
                html_body += "<TD>" + row.bookid + "</TD>";
                html_body += "<TD><B>" + row.title + "</B></TD>";
                html_body += "<TD>" + row.author + "</TD>";
                html_body += "<TD>" + row.category + "</TD>";
                html_body += "<TD>$" + Number(row.price).toFixed(2) + "</TD>";
```

```
                html_body += "</TR>";
            }

            html_body += "</TABLE>";
            html_body += "</BODY></HTML>";

            // Send query results back to the browser as HTML
            res.setHeader('Content-Type', 'text/html');
            return res.send(html_body);

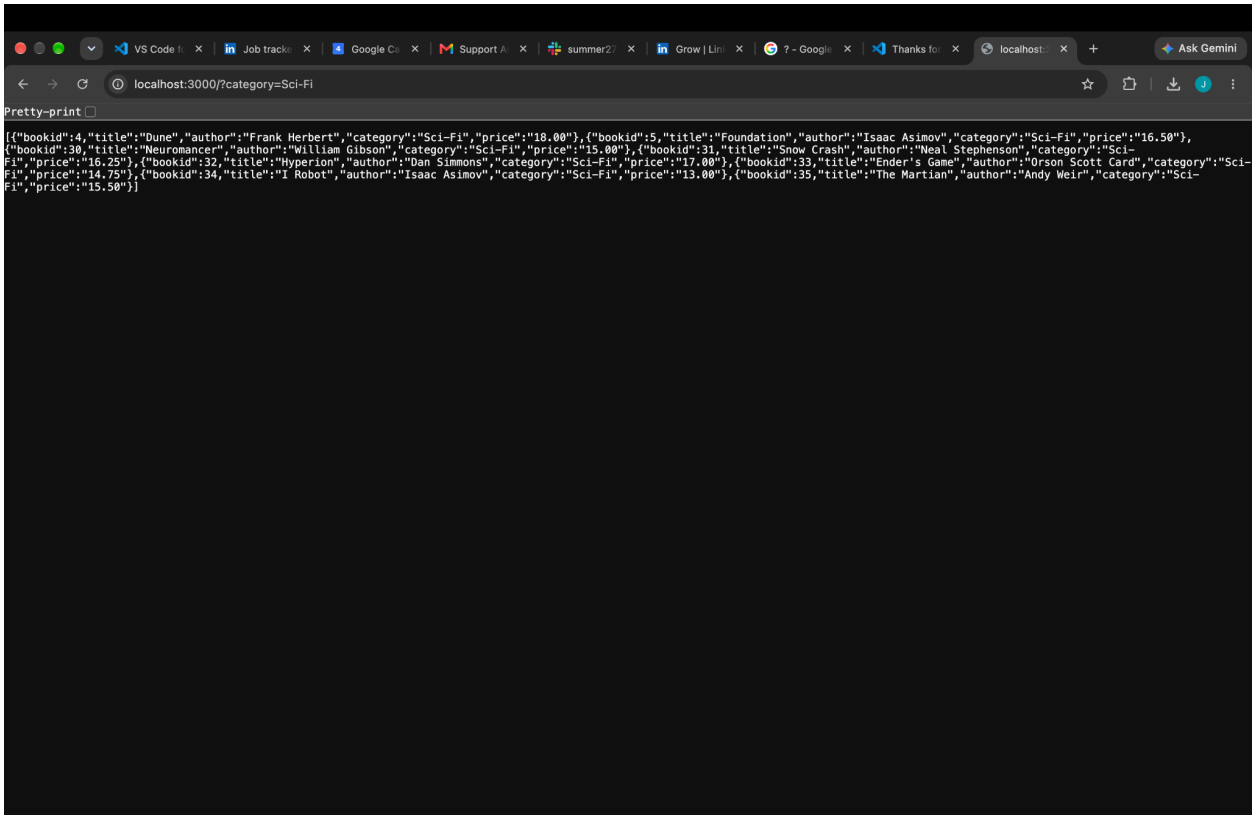
        } else {
            // Default directly to JSON format (matching Server 3 style)
            return res.status(200).json(result);
        }

    } catch (err) {
        // Catch and handle database errors cleanly (matching Server 3 style)
        console.error("Database error:", err.message);
        res.status(500).send("A database error occurred.");
    }
});

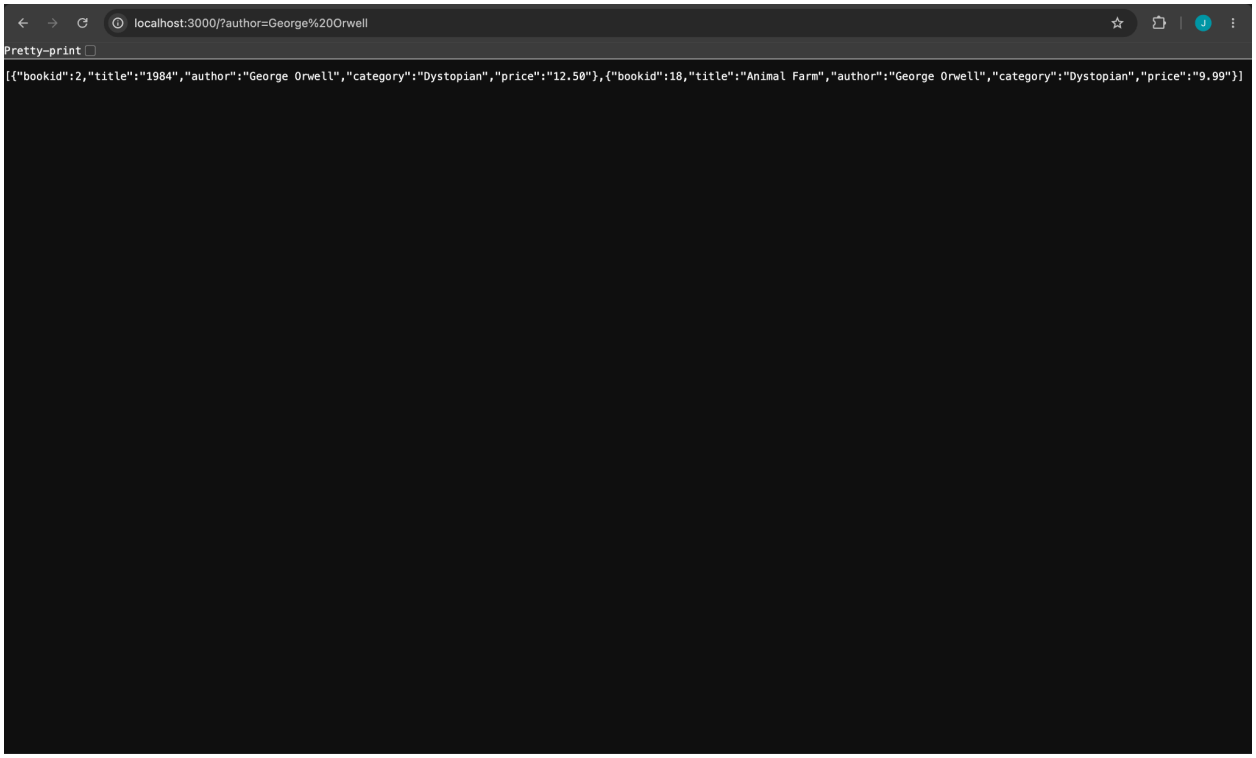
//*** server waits indefinitely for incoming requests
app.listen(port, function () {
    console.log("NodeJS app listening on port " + port);
});
```

JSON Examples:

URL: http://localhost:3000/?category=Sci-Fi



URL: http://localhost:3000/?author=George%20Orwell


localhost:3000/?author=George%20Orwell
Pretty-print
[{"bookid":2,"title":"1984","author":"George Orwell","category":"Dystopian","price":"12.50"},{"bookid":18,"title":"Animal Farm","author":"George Orwell","category":"Dystopian","price":"9.99"}]